Flood alert application using android

Presented By:
Rohit Iyer (510516084)
Parnavi Sen (510516011)
Sukrita Saha (510516082)
Mamon Almas (510516015)

under the supervision of
Prof. Ashish Kumar Layek

Introduction- the issue

- As the globe undergoes extreme climate changes, disaster events and their disaster scale continue to increase; therefore, it has become imperative to devise disaster prevention measures.
- The cases of flood related disasters have drastically increased especially in India with a dense population in coastal areas.

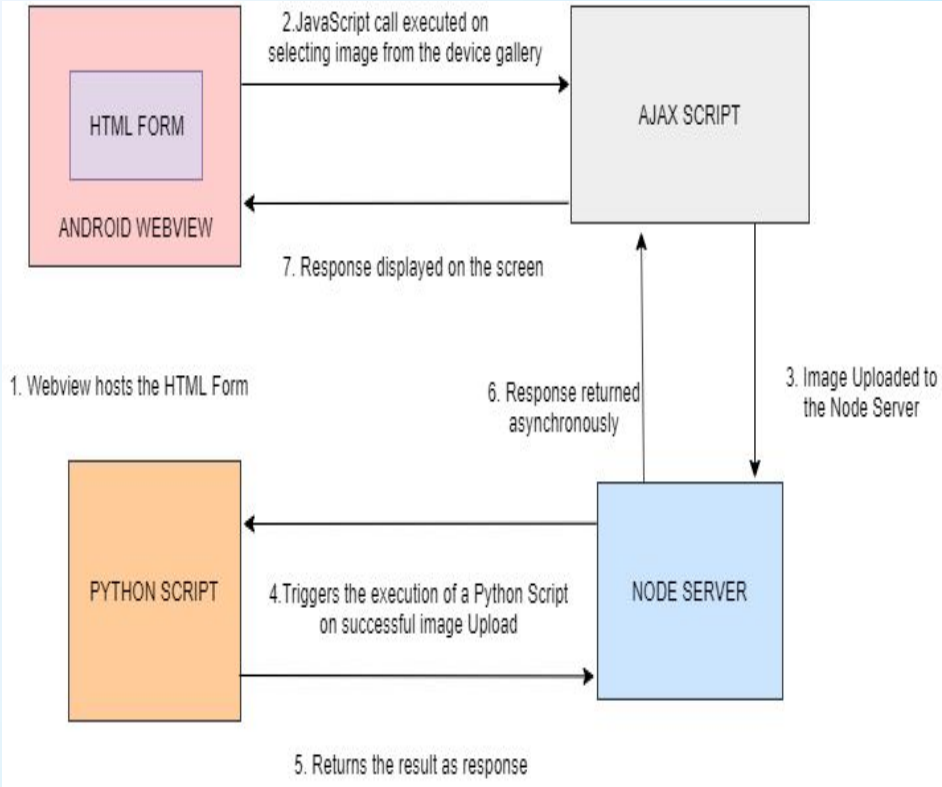
Motivation

- Destruction due to floods can only be mitigated by immediate response to crisis situation.
- With the support of digital platforms, the information can be quickly communicated to disaster management teams about the severity of inundation.
- In this project, we develop an android application which would take as input the sets of images belonging to flood affected areas from rescue personnels and filter them using deep learning approaches.
- The severity of floods is estimated and plotted on a map using the geolocations of the images.
- Subsequently the affected areas (also termed as **“hotspots“**) are discovered for immediate relief operations.

System design (1/2)

Framework:

- Android webview hosts the html form.
- Javascript call executed on button click.
- Server script spawns a child process.
- Python code executed in the backend.
- Backend code accesses the server database.
- Returns required data JSON format.



System design (2/2)

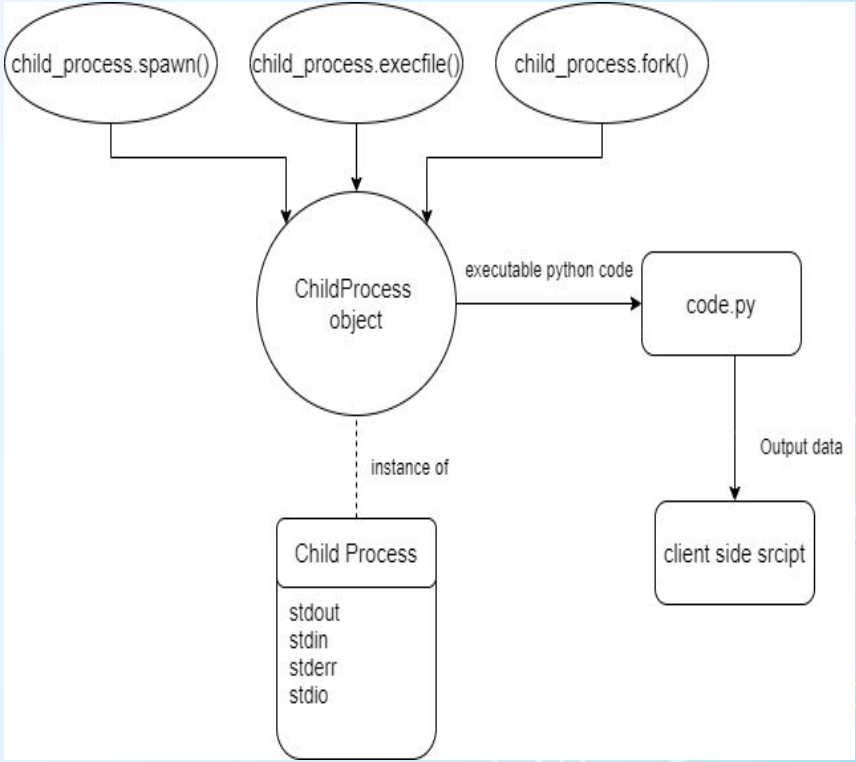
When multiple requests are sent to the server:

```
const {spawn} = require('child_process');
const child = spawn('python',["code.py",
arguments]);
```

The **spawn** method spawns an external application in a new process and returns a streaming interface for I/O . spawn returns a **ChildProcess** object.

As spawn returns a stream based object, it's great for handling applications that produce **large amounts of data**.

The python script performs the required task and returns the output as a response back to the server.



Backend script

The implementation of the backend code was done in python. There are two python scripts:

1. For training using the available datasets.
2. Prediction of images provided by rescue personnel.

Training Data:

1. **Generation of flood-water training samples**: To generate flood-water training samples, several flood Images are collected from www.google.com with the keyword pattern: " Flood", for example, "Orissa Flood", "Mumbai Flood" etc.

2. **Generation of non-water training samples**: To generate non-water samples, various keywords were used to download images, a few of them being: animals, cars, paintings, face, flowers, house, text, trees etc. However, we excluded any image having some water presence.

Non water image

Flood water image

Flood detector model

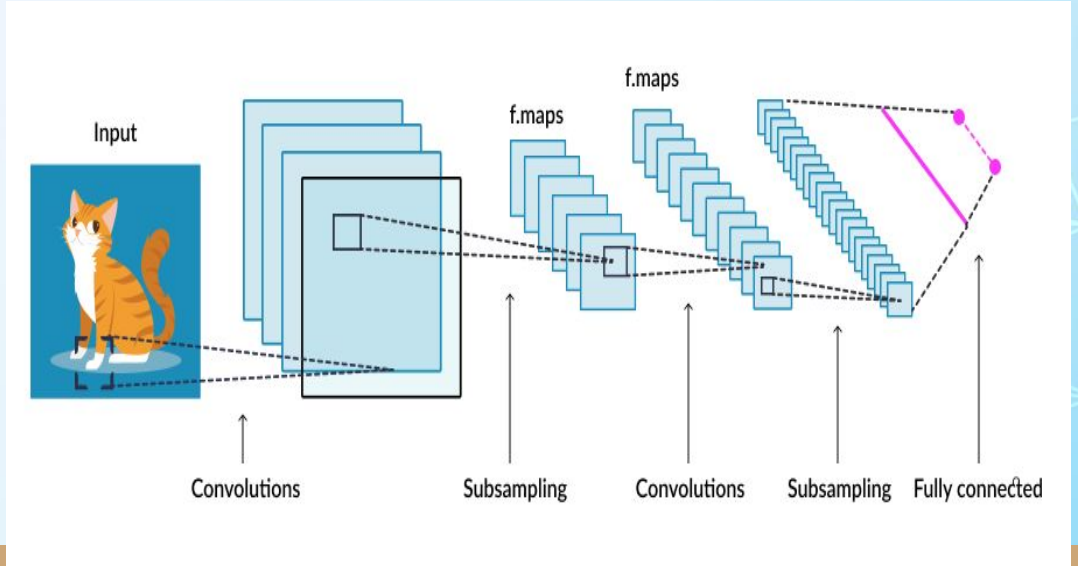
In our work, we used a CNN-based flood-water-detector model.

We use 2 approaches to create model:

1. Creating our own model for training and prediction.
2. Using transfer learning and performing layer unfreezing

Advantages of using CNN model:

1. Well adapted to classify images
2. Shareable weights
3. Translational invariant

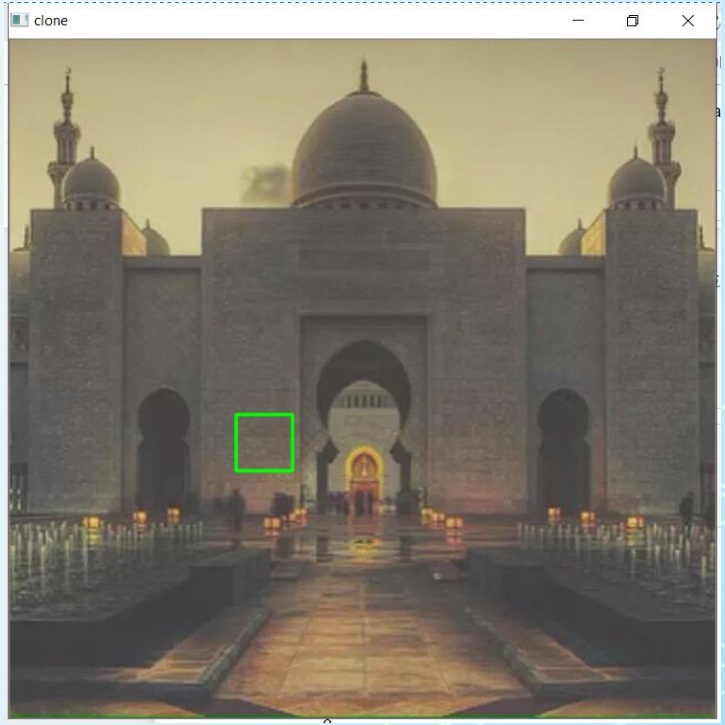


Generation of training dataset

- The proposed CNN model is trained using two different kinds of image samples:

  (i) flood-water and (ii) non-water.

- We use training sample dimension of 48 × 48 × 3. We run a non overlapping window of the same size over the entire set of images (both water and non water) and thus create dataset of images.
- The window size 48 × 48 x 3 is moved from left to right and top to bottom, and image under the window is cropped to produces several image patches.
- Total dataset size : 54k images of size 48 x 48 x 3.



Model created

Model with 4 Convolution and Max pooling layers followed by 2 fully connected layers and softmax activation at the output.

Activation function used are Rectifier Linear Unit.

Total no. of parameters: 4,980,482

Training Accuracy: 88.74%

Validation Accuracy: 88.23%

Serial No.	Layer Type	Filter Dimension	Number of Filters	Output Dimension
1	Input	-	-	48 x 48 x 3
2	Convolution + ReLu + Dropout (0.2) + Max Pooling	3 x 3 x 3 2 x 2	64	48 x 48 x 64 24 x 24 x 64
3	Convolution + ReLu + Dropout (0.2) + Max Pooling	3x 3 x 3 2 x 2	128	24 x 24 x 128 12 x 12 x 128
4	Convolution + ReLu + Dropout (0.2) + Max Pooling	3x 3 x 3 2 x 2	256	12 x 12 x 256 6 x 6 x 256
5	Convolution + ReLu + Dropout (0.2) + Max Pooling	3 x 3 x 3 2 x 2	512	6 x 6 x 512 3 x 3 x 512
6	Fully Connected (64) Dropout (0.2)	-	-	1 x 1 x 64
7	Fully Connected (2) Softmax Output	-	-	1 x 1 x 2 1 x 1 x 2

Transfer learning

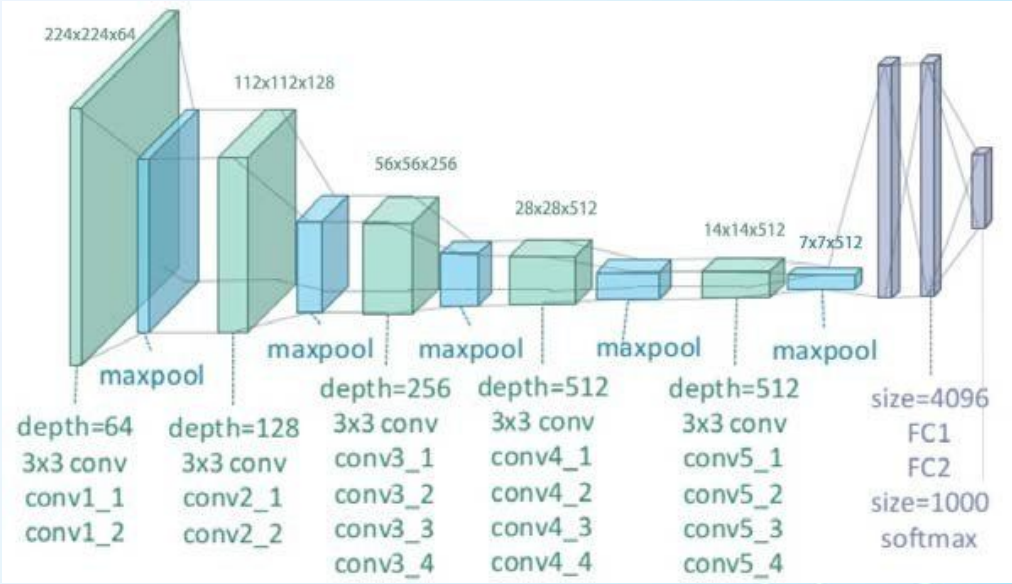
- With transfer learning, instead of starting the learning process from scratch, we start from patterns that have been learned when solving a different problem.
- Features computed by the last layer are specific (problem dependent) and depend on the chosen dataset and task.
- While using a pre-trained model remove the original classifier, we add a new classifier that fits your purposes.
- One of the **important features** of transfer learning is that we only load the weights at each layer of the pretrained model which are independent of the size of the input image. Thus images of any size can be trained using the model.
    - e.g VGG19 model usually takes input image of size 224 x 224 x 3
    - However in our case, our input images are of size 48 x 48 x 3.

Pretrained model

We used the VGG19 pretrained model in our application as the depth of the network is a critical component for good performance.

The VGG19 model consists of 19 weight layers (16 convolution layers and 3 fully connected layers).

Number of parameters ~ 20 million

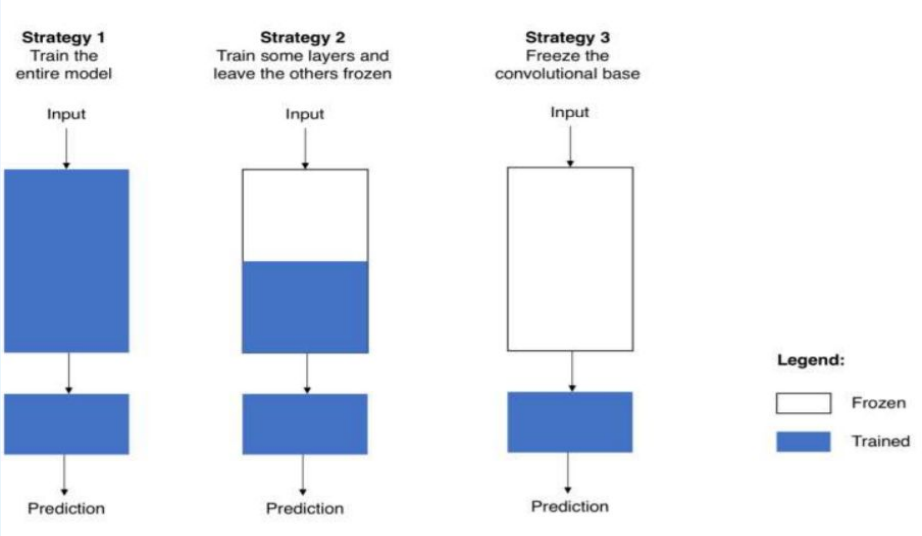


Strategies of transfer learning

There are 3 ways of using transfer learning:

1. We use the architecture of the pre-trained model and train it according to your dataset.
2. Train some layers and leave the others frozen.
3. Freezing the entire convolutional base.

All these 3 methods were experimented with.



Experimental approaches

Description	Freezing entire convolutional base	Some layers unfrozen, all others unfrozen	All the layers Unfrozen
Properties	Might lead to underfitting Used when data is very similar to benchmark data used in pre trained model.	Playing with the Dichotomy	Might lead to overfitting. Used when data is not similar to benchmark data used in pre trained model.
Model Summary	Total params: 20,090,306 Trainable params: 0 Non-trainable params: 20,090,306	Total params: 20,090,306 Trainable params: 9,505,154 Non-trainable params: 1,05,85,152 (Layers After Convolution layer block_5 conv_1 are unfrozen)	Total params: 20,090,306 Trainable params: 20,090,306 Non-trainable params: 0

How layers are unfreezed

1. flag=False
2. vgg_model.trainable = True
3. for layer in vgg_model.layers:
4. if layer.name=='block5_conv1':
5. flag=True
6. layer.trainable = flag

False → Layer Freezed

True → Layer Unfreezed

	Layer Name	Layer Trainable
0	input_3	False
1	block1_conv1	False
2	block1_conv2	False
3	block1_pool	False
4	block2_conv1	False
5	block2_conv2	False
6	block2_pool	False
7	block3_conv1	False
8	block3_conv2	False
9	block3_conv3	False
10	block3_conv4	False
11	block3_pool	False

	Layer Name	Layer Trainable
12	block4_conv1	False
13	block4_conv2	False
14	block4_conv3	False
15	block4_conv4	False
16	block4_pool	False
17	block5_conv1	True
18	block5_conv2	True
19	block5_conv3	True
20	block5_conv4	True
21	block5_pool	True
22	flatten_20	True

Results obtained from various scenarios

Freezing entire convolutional base				Some layers unfrozen, all others unfrozen				All the layers Unfrozen			
Class Label	Precision	Recall	F - score	Class Label	Precision	Recall	F - score	Class Label	Precision	Recall	F - score
0 (non water)	0.99	0.98	0.99	0 (non water)	0.99	0.99	0.99	0 (non water)	0.99	0.98	0.99
1 (water)	0.89	0.89	0.89	1 (water)	0.91	0.89	0.90	1 (water)	0.89	0.95	0.92
Accuracy: 94%				Accuracy: 97.7%				Accuracy: 98.2%			
Model provides least accuracy among the three.				Appreciable accuracy with good performance on test data as wells as prediction				High training and test accuracy but performs moderately in prediction and image binarization.			

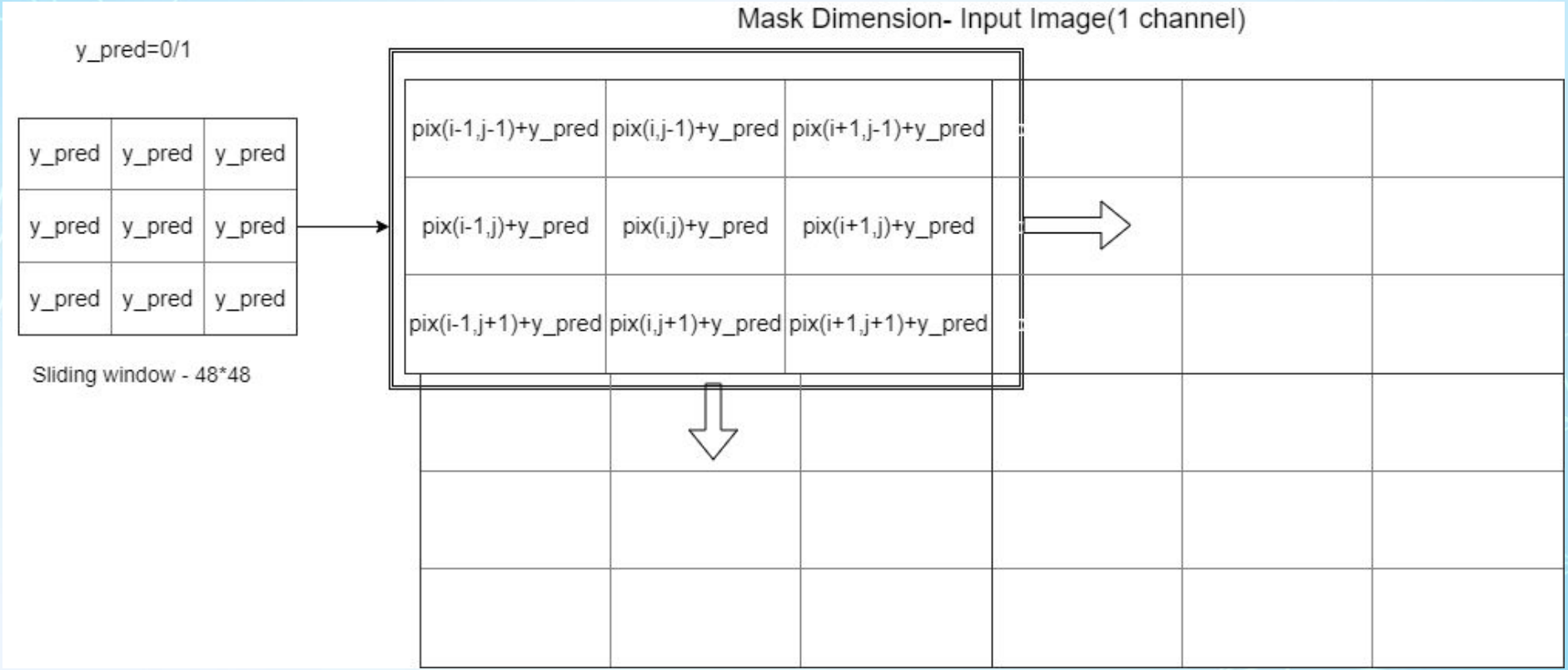
Response matrix generation (heatmap)

Overlapping sliding window approach to generate the response matrix for the input image.

As a preprocessing step, the input image is zero padded in all the sides of the images. 48 zeros are padded on the four sides of the image under test.

Steps for Response Matrix Generation:
i. A Matrix named mask is prepared of the same dimensions of the input image (only 1 channel) (48 x 48 x 1) containing zeros.
ii. Each image patch (48 x 48 x 3) is fed to the flood-water-detector model. The output is a prediction of value 1 (water) or 0 (non water).
iii. This value is added to all 48 x 48 cells in the response matrix mask

Algorithm for response matrix generation (1/2)



Algorithm for response matrix generation (2/2)

First, we remove the padding added

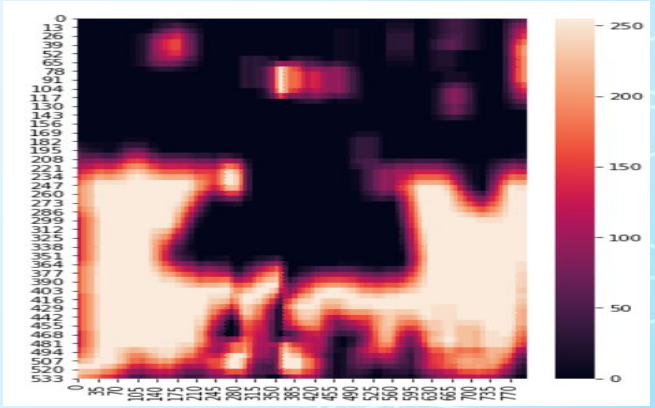
Algorithm:

Input Image →

mask[y:y + winW, x:x + winH]=
mask[y:y + winW, x:x + winH]+ np.full((48,48), y_pred)

where,
winW = winH =48.
y_pred = 0 if image patch = non water
y_pred = 1 if image patch = water

Heatmap →



Flood region identification (1/2)

First the response matrix is divided by the maximum value in the response matrix .

mask[i,j]=mask[i,j] / maxval

Then we convert the value in the matrix mask to 0 or 255 depending upon a threshold:

img[i,j] = 0 if mask[i,j] <= T

img[i,j] = 255 if masl[i,j] > T

T → Threshold

0<T<1

Region covered by water: After the image binarization, the percentage of image covered by water is calculated as follows:

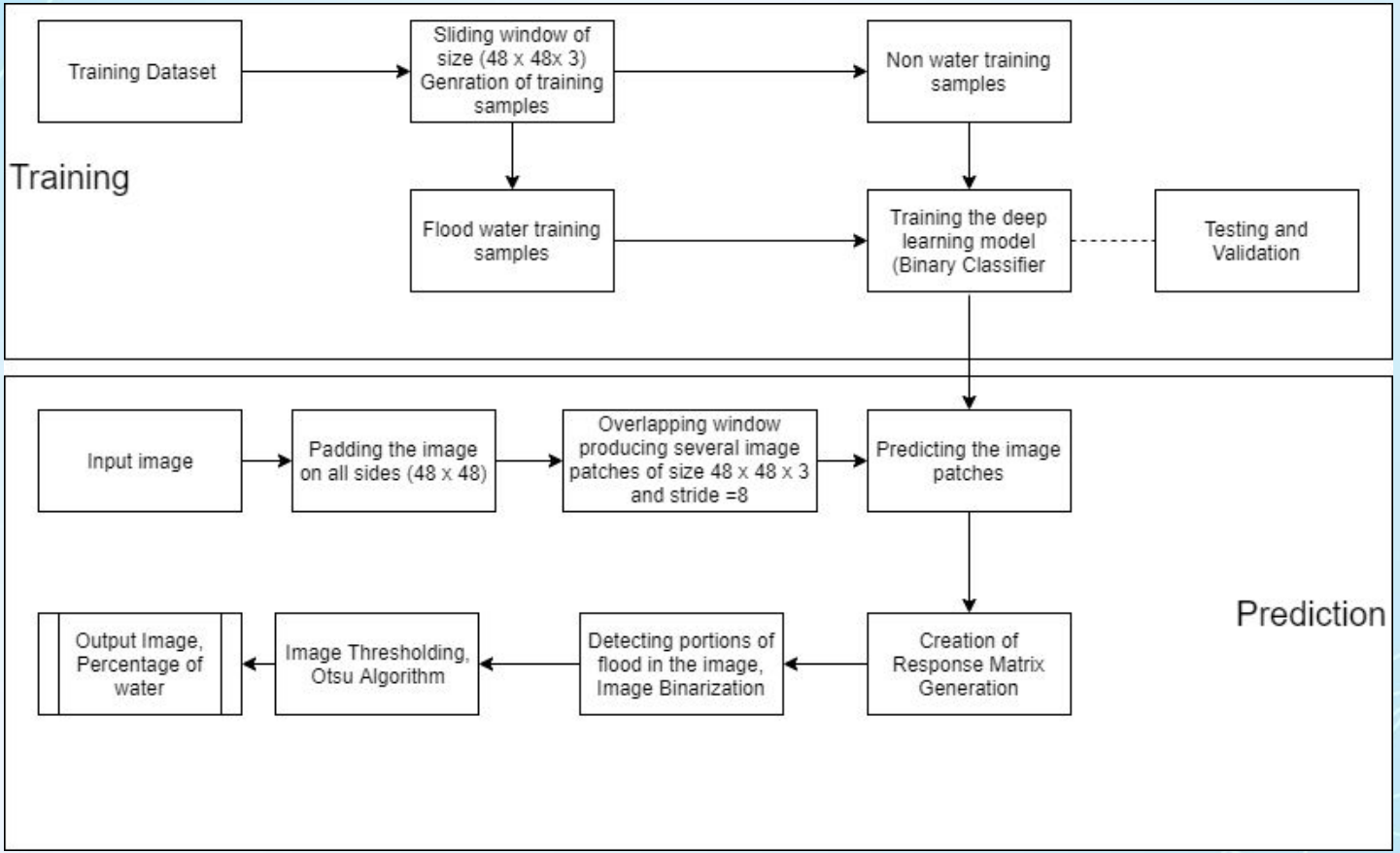
$$\text{\% of image covered by water} = \frac{\text{No. of water pixels}}{\text{Total no. of pixels}} * 100$$

Flood region identification (2/2)

We also used the Otsu Algorithm for the image binarization which is a thresholding method for image binarization.
The algorithm divides the image histogram into two classes, by using a threshold such as the in-class variability is very small.

Function g(x, y) can be defined over every pixel value of the
original image f(x, y)
that defines the new threshold image.
g(x, y) = 0 x<T
1 x ≥ T
Where the threshold is T.

Flowchart


Training Dataset
Sliding window of size (48 x 48x 3) Genration of training samples
Non water training samples
Training
Flood water training samples
Training the deep learning model (Binary Classifier
Testing and Validation
Input image
Padding the image on all sides (48 x 48)
Overlapping window producing several image patches of size 48 x 48 x 3 and stride =8
Predicting the image patches
Output Image, Percentage of water
Image Thresholding, Otsu Algorithm
Detecting portions of flood in the image, Image Binarization
Creation of Response Matrix Generation
Prediction

Frontend

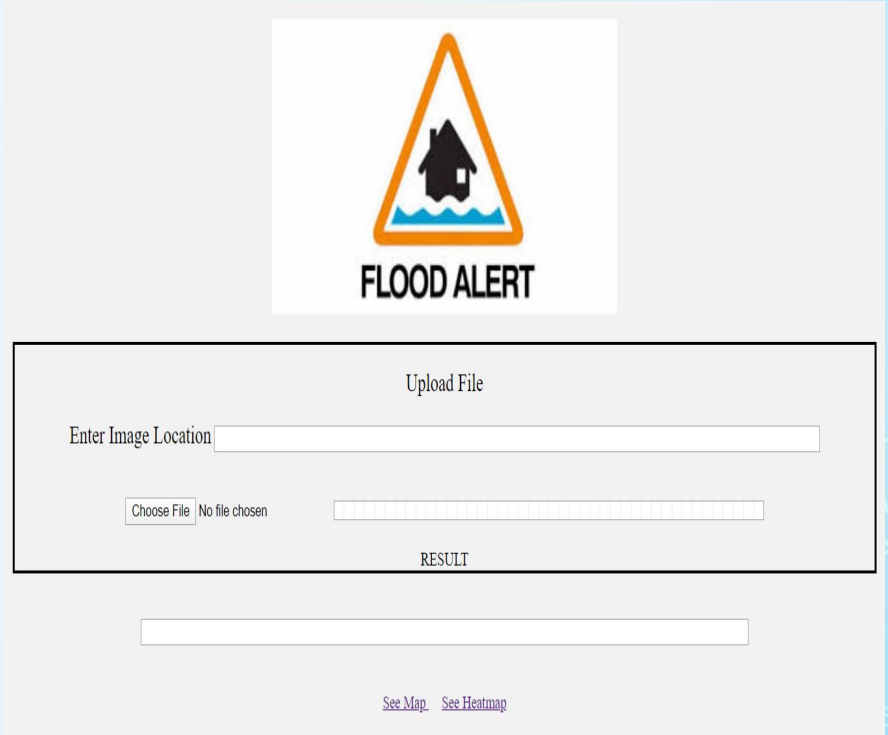
The android app has been prepared using Android Studio and the Nodejs Server is used for storing the data and running the script for detection of flood images.

Features of the application:

1. Uploading image to server:- Returns the result as water percentage and provided to the user.

2. Viewing affected areas on map (or heatmap) of a desired city.

3. View an image on the map: - By clicking the desired marker on the map. Returns the image in an infobox



Simulation

Web Scraping:

Selenium webdriver Python API to emulate human browsing the website on the web browser (Chrome) and scroll down till the end of the album page.

Steps for web scraping images:

1. The search term (e.g. “Mumbai Floods”) and the number of images (e.g. 500) need to be entered.
2. An automated web browser with the given search term is opened and all the google images are loaded.
3. The browser automatically starts to scroll down one-page height at a time until the end of the page has been reached and thus, loads all the thumbnails.
4. We use “css-selectors” to get the relevant elements from the page i.e all the image links.
5. Following this, the automated web browser tries to click every thumbnail (img.click()) such that it can get the real image behind it.
6. A directory is created to save all the images.

Assigning coordinates to scrapped images

Algorithm for selecting coordinates:
In order to simulate a real world scenario, we identified certain regions in cities that are low lying areas or are prone to floods. Example in Mumbai the areas of Andheri, Kurla, Juhu, etc were identified. Coordinates of these locations were taken as cluster centres.
Each cluster centre is assigned a scrapped image and other images are distributed randomly to each of the clusters.
Coordinates for the images were assigned randomly within 2km radius of the cluster centre. This algorithm, is just for a simulated scenario and is based on ideal conditions.

```
cluster_size=random.randrange(10,150,1) # Lower limit =10, Upper limit=150
```

$$covariance = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$ # covariance matrix

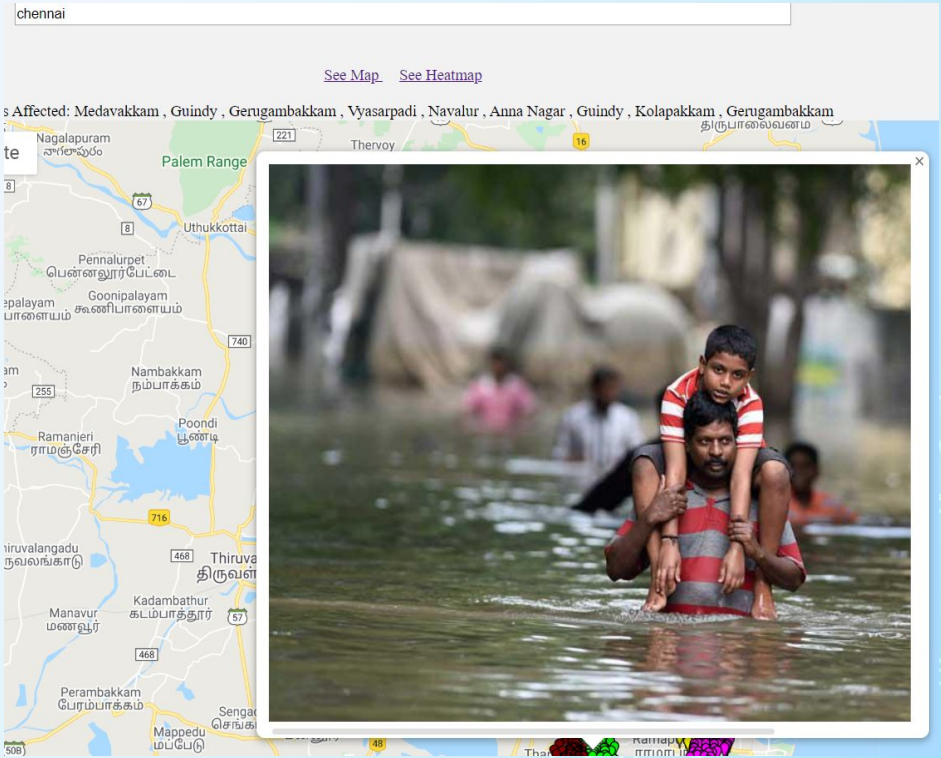
```
x,y = np.random.multivariate_normal(cluster_center,covariance,cluster_size)
```

$$img\_coodinates = cluster\_center\_coordinates - [(2km) \quad (2km)] + ([4Km] * [x \quad y])$$

where, 4 km is the diameter.

Working with the simulated scenario

- All the images which have flood water content less than a specified threshold are removed.
- The deep learning model is used to calculate the water percentage which has been explained previously in detail.
- All the images are plotted on terrain maps using the “ Maps JavaScript API “.
- Markers are added for each image location.
- Each marker has an on click event which opens the image in an infobox and its details.



Extraction of geolocation from the metadata of image

- Image metadata is text information pertaining to an image file that is embedded into the file.
- Image metadata includes details relevant to the image itself as well as information about its production such as the GeoInfo.
- With photos stored in jpg and many other file formats, the geotag information, camera location and sometimes heading, is typically embedded in the metadata, stored in Exchangeable image file format (Exif) or Extensible Metadata Platform (XMP) format.

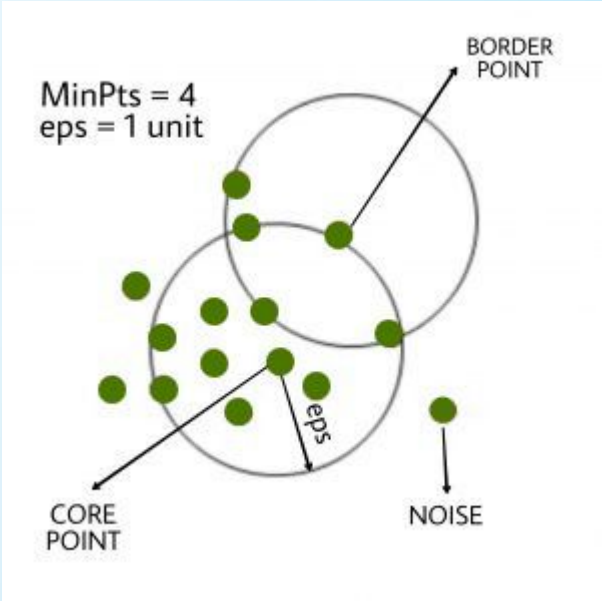
GPSAltitude	tuple	2	(76233, 1000)
GPSAltitudeRef	bytes	1	
GPSDateStamp	str	1	2019:05:15
GPSLatitude	tuple	3	((19, 1), (15, 1), (157374, 10000))
GPSLatitudeRef	str	1	N
GPSLongitude	tuple	3	((72, 1), (59, 1), (24252, 10000))
GPSLongitudeRef	str	1	E
GPSProcessingMethod	str	1	ASCII GPS
GPSTimeStamp	tuple	3	((13, 1), (11, 1), (34, 1))

Clustering (1/2)

- The most suitable algorithm for clustering when the number of cluster centres and the shape of cluster is not known is the **DBSCAN algorithm**.

- DBSCAN groups together points that are close to each other based on a distance measurement and a minimum number of points. It also marks as outliers the points that are in low-density regions. Parameters in DBSCAN algorithm are MinPts, eps , distance measure.

- The DBSCAN algorithm gives the total number of estimated clusters. Using this we further implemented the **k - means algorithm** which gives the cluster centers.



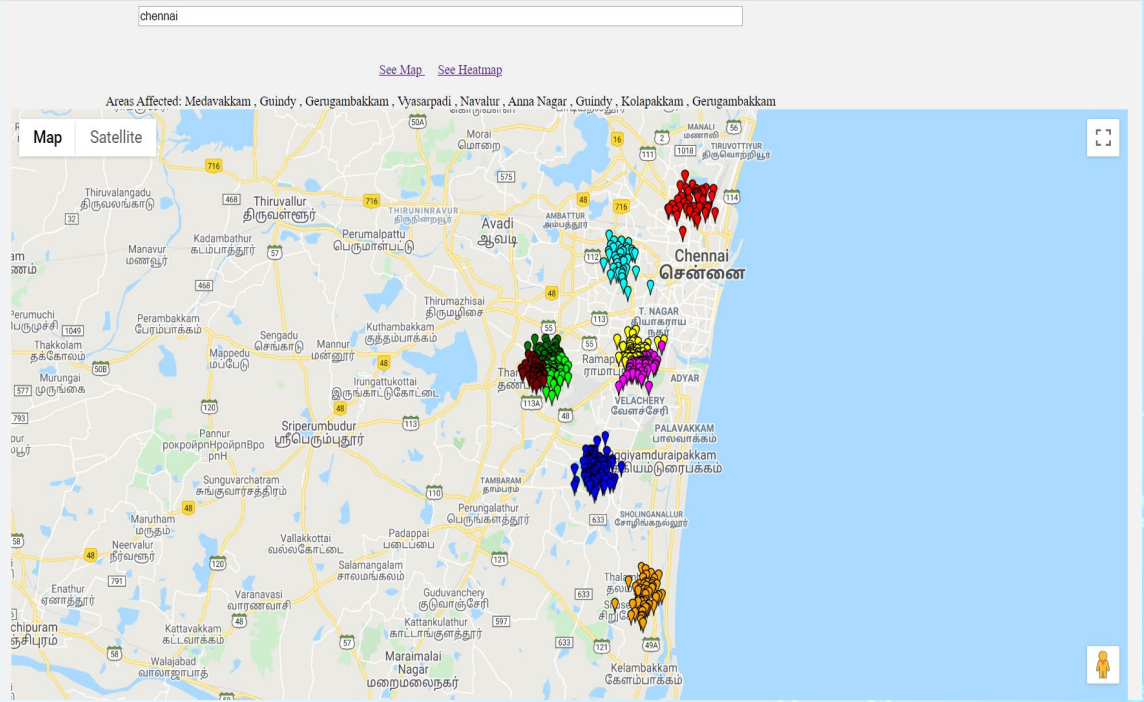
Clustering (2/2)

Parameters Used:

- Distance measure = ‘Haversine’
- Epsilon = 2 (km) /R
- min_samples=2

- Algorithm used for DBSCAN is ‘ball tree’.

where R =6373 km is the radius of earth

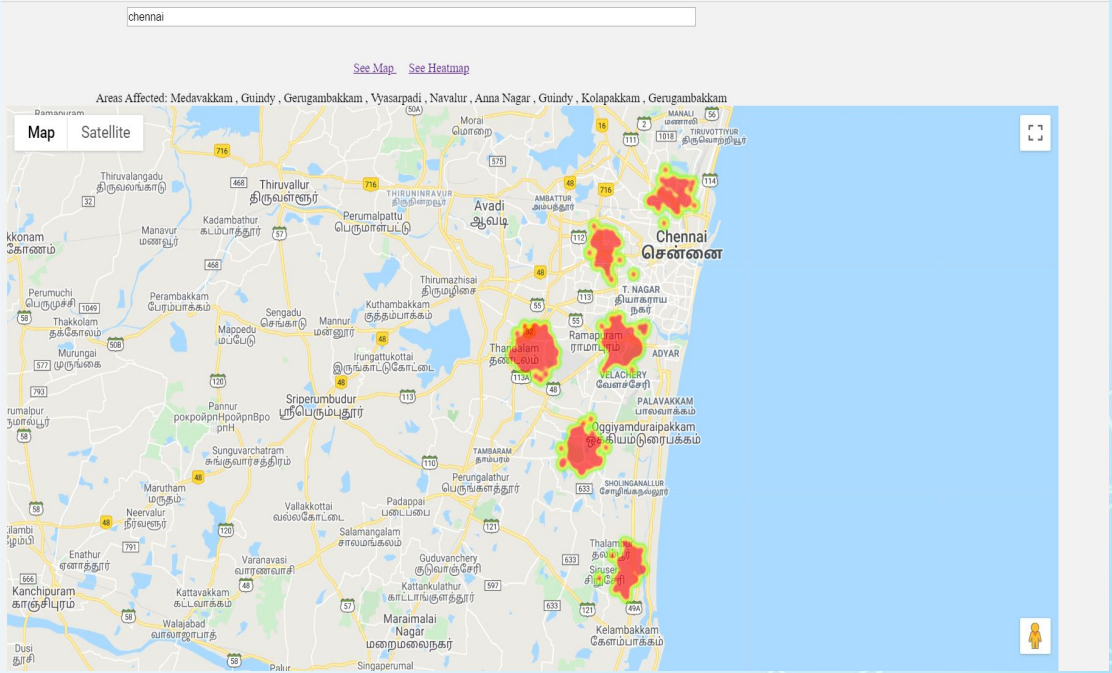


Clusters of affected zones in Chennai

Generating heatmap

- A heatmap is a visualization used to depict the intensity of data at geographical points. Areas of higher intensity will be colored red, and areas of lower intensity will appear green.

- Heatmap layer has been implemented in our project using the **google.maps.visualization** library provided by google maps.



Heatmap of affected zones in Chennai

Identifying affected localities

- Coordinates of the cluster centres can be retrieved after applying the k - means clustering on the set of images. These coordinates are helpful in order to find the center of the affected regions.

- The **Geocoding API** is a service that provides geocoding and reverse geocoding of addresses.

Cluster centres in Kolkata

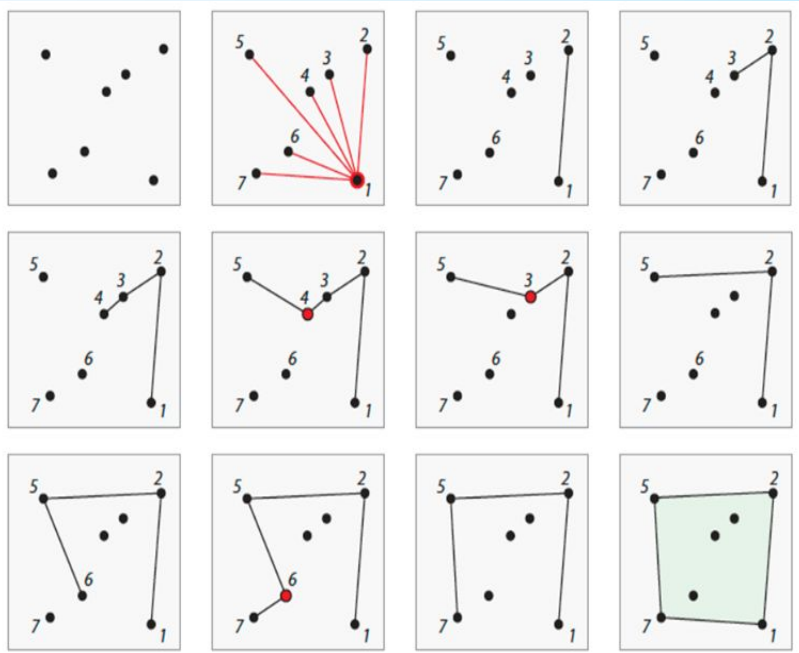
Generating boundaries to identify “hotspots”

- By generating boundaries to the existing clusters of affected regions we can define certain regions as **“ hotspots ”** which can be immediately cordoned off so as to allow only rescue personnels, volunteers and health workers to access the area and provide relief support to the gravely affected residents of the area.

- Boundaries are created with the Graham Scan algorithm which creates the smallest polygon encompassing all the coordinates.
- **Convex Hull:** For a given set of points S, the convex hull, denoted as CH(S), is the intersection of all convex regions containing S.Graham scan algorithm is used to find the convex hull of a set of points.

Graham scan algorithm

Choosing the bottommost point as an anchor , all other points are ordered based on the angles they form about this anchor. The hull is constructed by following this ordering, adding points for left hull turns and deleting for right turns.



Algorithm 1 Graham scan algorithm

```
1: i ← 2 , j ← 3 , k ← 4
2: while k ≤ n do
3:     if P_i P_j P_k is a right turn then
4:         while P_i P_j P_k is a right turn do
5:             Remove edges P_i P_j and P_j P_k
6:             Add edge P_i P_k
7:             j = k , k = k + 1
8:         end while
9:     else
10:         Add edge P_j P_k as hull edge
11:        i = j , j = k , k = k + 1
12:    end if
13: end while
```

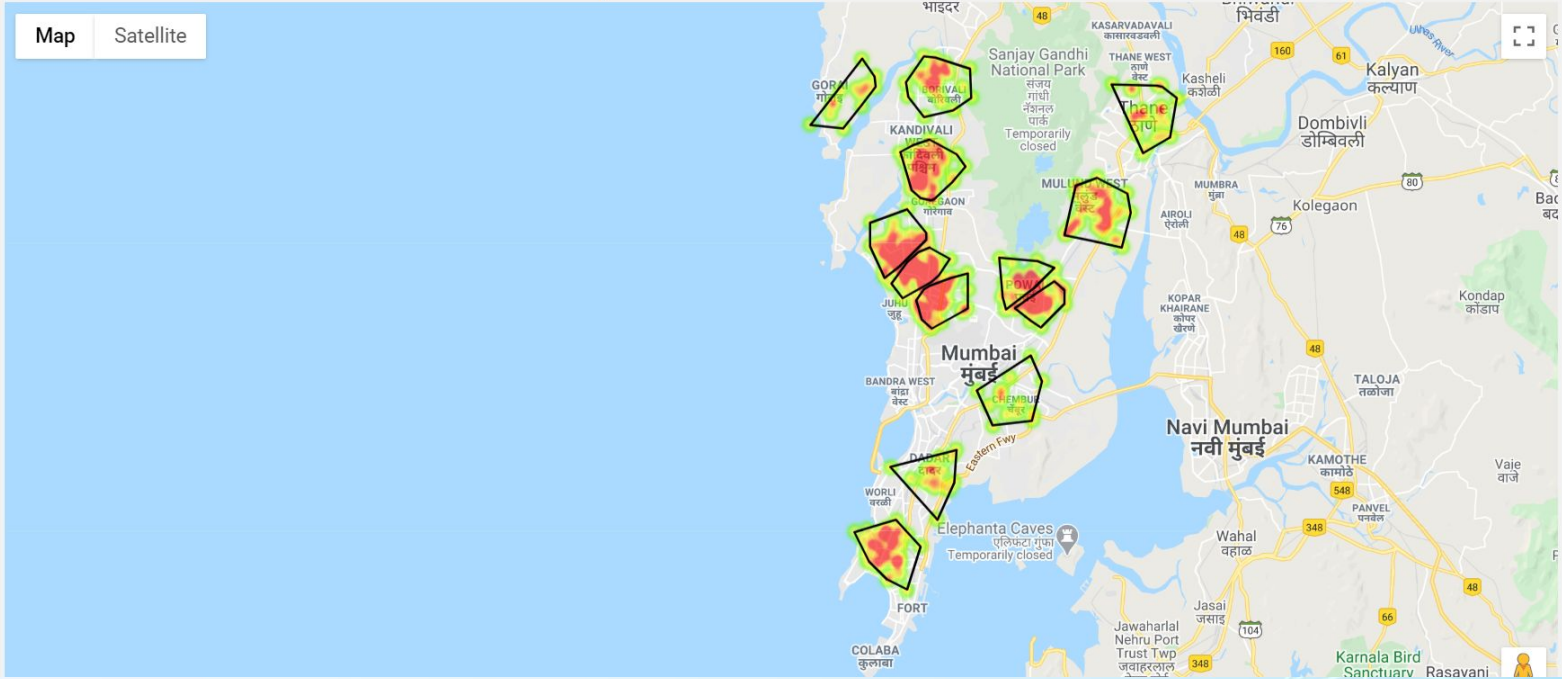
The algorithm takes O(nLogn) time if we use a O(nLogn) sorting algorithm

Hotspots identified in the city of Mumbai

mumbai

See Map See Heatmap

Areas Affected: Mumbai Central , Malad West , Mulund West , Andheri East , Thane , Powai , Andheri West , Borivali West , Chembur , Vikhroli West , Dadar , Gorai , Andheri W



Results and future scope

1. The model developed is very accurate (95%) in detecting water and non water regions in the image.
2. The deep learning approach to calculate the percentage of flood water in the image effectively filters out unwanted and wrong images that have been uploaded to the application.
3. Citizens travelling across the city and public transport can be made cautious to restrain themselves from travelling to such disaster affected regions.
4. The localities identified as ‘ hotspots ‘ would quickly alert the rescue personnels and civic volunteers to carry out relief operations in those zones.
5. Over a period of time the data can be analyzed and certain flood prone “hotspots” can be identified which need robust infrastructure, road maintenance and **storm water drainage** facilities.
6. Strategic location of emergency shelter homes can be found out,
7. Such data would be very useful for municipal corporations and NDRF teams working during crisis periods.

Challenges

1. The results obtained using the deep learning models had a very good training accuracy as well as test accuracy. However, it performed moderately in image segmentation. Thus there is a need for performing other image segmentation and preprocessing techniques and a comparative study between different techniques.
2. Certain portions in the water which have reflections or shadows, are treated as non water objects. There is scope for improvement in this aspect.
3. Only those images having locations in their metadata can be accepted by the application.
4. Deliberately manipulating image metadata and uploading fake images remains a challenge.
5. To make the application useful, there must be general awareness among the public and eagerness to contribute towards the benefit of the society.

References

1. Detection of Flood Images Posted on Online Social Media for Disaster Response Ashish Kumar Layek∗ , Soham Poddar† , Sekhar Mandal‡ Dept. of Computer Science & Technology, Indian Institute of Engineering Science & Technology, Shibpur, Howrah, India Email: ∗ ashish@cs.iiests.ac.in, † sohampoddar26@gmail.com , ‡ sekhar@cs.iiests.ac.in
2. Flood Alerts System with Android Application Mohamad Nazrin Napiah Department of Computer System & Technology Faculty of Computer Science & Information Technology University of Malaya nazrin23@siswa.um.edu.my
3. Automated Image Identification Method for Flood Disaster Monitoring In Riverine Environments: a Case Study in Taiwan Jyh-Horng Wu, Chien-Hao Tseng, Lun-Chi Chen, Shi-Wei Lo, Fang-Pang Lin National Center for High-Performance Computing, National Applied Research Laboratories, Taiwan
4. FLOOD-WATER LEVEL ESTIMATION FROM SOCIAL MEDIA IMAGES P. Chaudhary1 , S. D'Aronco1 , M. Moy de Vitry2 , J. P. Leitao˜ 2 , J. D. Wegner1
5. https://developers.google.com/maps
6. https://towardsdatascience.com/image-scraping-with-python-a96feda8af2d
7. https://towardsdatascience.com/a-comprehensive-hands-on-guide-to-transfer-learning-with-real-world-applications-in-deep-learning-212bf3b2f27a
8. https://medium.com/kansas-city-machine-learning-artificial-intelligen/an-introduction-to-transfer-learning-in-machine-learning-7efd104b6026
9. https://whatis.techtarget.com/definition/image-metadata

THANK YOU